

EXERCICE 1

1. Centrer un champ de texte dans un conteneur

Méthode 1 : Utiliser « center » dans « alignement » du Container

On peut centrer le « TextField » en réglant « alignement : Alignment.center » dans le **Container**.
Le Container prend toute la place disponible et son enfant est centré.

Exemple :

```
Container (  
  alignment : Alignment.center ,  
  color : Colors.grey[200] ,  
  height : 200 ,  
  child : TextField(  
    decoration : InputDecoration(hintText : 'BIENVENUE '),  
  ),  
)
```

Argument : simple, rapide, centrage horizontal et vertical. Idéal quand le Container a une taille définie.

Méthode 2 : Utiliser « Center » comme widget parent

On encapsule le « TextField » dans un widget « Center ». Le « Center » occupe tout l'espace donné et centre son enfant.

Exemple :

```
Center (
  child : SizedBox(
    width : 200,
    child : TextField(
      decoration : InputDecoration(hintText : 'BIENVENUE',
    ),
  ),
)
```

Argument : Plus explicite, plus flexible si on veut centrer uniquement horizontalement ou verticalement. Très utilisé dans des layouts complexes.

2. Ajouter un compteur de caractères à « counterText »

Méthode 1 : Utiliser buildCounter dans InputDecoration

On peut personnaliser l’affichage du compteur avec buildCounter.

Exemple :

```
TextField(
  maxLength: 50,
  decoration: InputDecoration(
    labelText: 'Ici le message ! ',
    counterText: ",
    buildCounter: (context, {required currentLength, required isFocused, maxLength}) {
      return Text('$currentLength / $maxLength');
    },
  ),
)
```

Méthode 2 : Utiliser « counterText » directement

On modifie la propriété `counterText` pour afficher un message personnalisé.

Exemple :

```
TextField(  
  maxLength: 50,  
  decoration: InputDecoration(  
    labelText: 'Votre message',  
    counterText: 'Caractères saisis : 0/50',  
  ),  
  onChanged: (value) {  
    // Mettre à jour counterText dynamiquement (plus complexe, nécessite un StatefulWidget)  
  },  
)
```

Argument : La première méthode est plus propre et automatique. La seconde demande plus de travail mais offre plus de contrôle sur le style.

3. Additionner deux nombres avec des contrôleurs et parse

Exemple étape par étape :

- a. Créer deux `TextEditingController`.
- b. Récupérer les textes des champs.
- c. Convertir avec `int.parse` ou `double.parse`.
- d. Additionner et afficher le résultat.

```

final controller1 = TextEditingController();
final controller2 = TextEditingController();
int resultat = 0;
// Dans un ElevatedButton :
void additionner() {
    int nb1 = int.parse(controller1.text);
    int nb2 = int.parse(controller2.text);
    setState(() {
        resultat = nb1 + nb2;
    });
}

```

Argument : `TextEditingController` permet de lire et modifier le texte facilement. `parse` convertit la chaîne en nombre après validation.

4. Utiliser decoration dans un TextField

Exemple :

```

TextField(
  decoration: InputDecoration(
    labelText: 'Nom complet',
    hintText: 'Entrez votre nom',
    prefixIcon: Icon(Icons.person),
    suffixText: 'obligatoire',
    border: OutlineInputBorder(),
    filled: true,
    fillColor: Colors.white,
  ),
)

```

Argument : « decoration » permet d'améliorer l'apparence et l'ergonomie : icônes, bordures, labels, indicateurs, marges intérieures.

EXERCICE 3

Oui, c'est possible.

Il faut utiliser une Column comme enfant du Container.

Exemple fonctionnel :

```
Container(  
  padding: EdgeInsets.all(16),  
  decoration: BoxDecoration(  
    color: Colors.white,  
    borderRadius: BorderRadius.circular(12),  
    boxShadow: [BoxShadow(blurRadius: 5)],  
  ),  
  child: Column(  
    children: [  
      TextField(decoration: InputDecoration(labelText: 'Nom')),  
      SizedBox(height: 10),  
      TextField(decoration: InputDecoration(labelText: 'Prénom')),  
      SizedBox(height: 10),  
      TextField(decoration: InputDecoration(labelText: 'Email')),  
    ],  
  ),  
)  
)
```

Argument :

Un Container peut contenir n'importe quel widget. En utilisant Column, on empile plusieurs TextField verticalement. Cela permet de regrouper visuellement des champs de saisie dans une même zone.